

Parsing

Context-Free Grammar & CKY algorithm

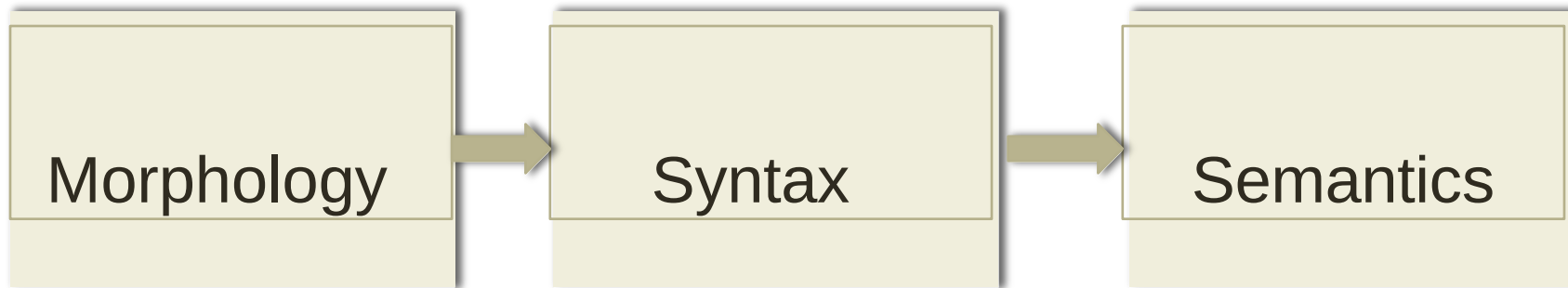
What is Syntax?

- Study of structure of language
- How words are arranged in a sentence and the relationship between them.

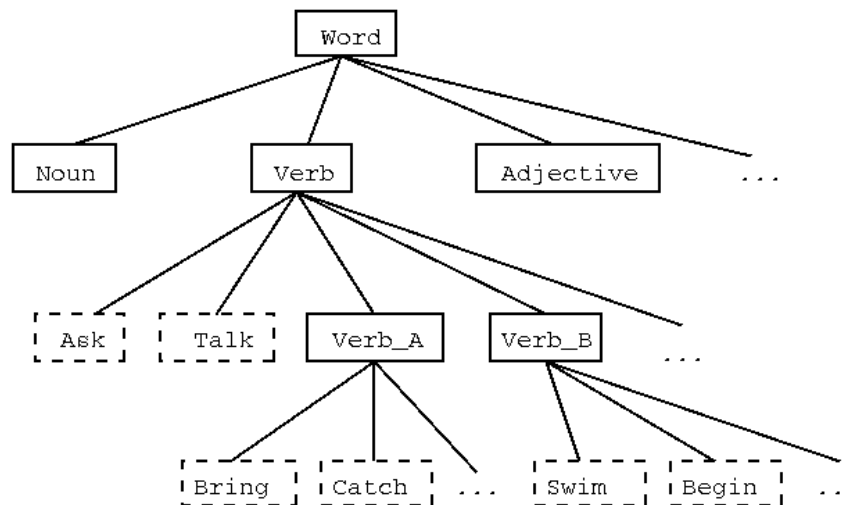
What Syntax is Not

- Phonology: study of sound systems and how sounds combine
- Morphology: study of how words are formed from smaller parts (morphemes)
- Semantics: study of meaning of language

Syntax as an interface



Representational
Device



Need for Syntax

- Grammar checkers
- Question answering
- Information extraction
- Machine translation

Natural language syntax

Constituents

Dependencies

Context-free grammar

Arguments and modifiers

views of linguistic structure:

Constituency (phrase structure)

Phrase structure organizes words into nested constituents.

How do we know what is a **constituent**

Distribution: a constituent behaves as a unit that can appear in different places:

John talked [to the children] [about drugs].

John talked [about drugs] [to the children].

*John talked drugs to the children about

The rise of annotated data: The Penn Treebank

```
( (S
  (NP-SBJ (DT The) (NN move))
  (VP (VBD followed)
    (NP
      (NP (DT a) (NN round))
      (PP (IN of)
        (NP
          (NP (JJ similar) (NNS increases))
          (PP (IN by)
            (NP (JJ other) (NNS lenders))))
          (PP (IN against)
            (NP (NNP Arizona) (JJ real) (NN estate) (NNS loans)))))))
  ( , , )
  (S-ADV
    (NP-SBJ (-NONE- *))
    (VP (VBG reflecting)
      (NP
        (NP (DT a) (VBG continuing) (NN decline))
        (PP-LOC (IN in)
          (NP (DT that) (NN market))))))
  ( . . )))
```


The rise of annotated data

Starting off, building a treebank seems a lot slower and less useful than building a grammar

But a treebank gives us many things

- Reusability of the labor

 - Many parsers, POS taggers, etc.

 - Valuable resource for linguistics

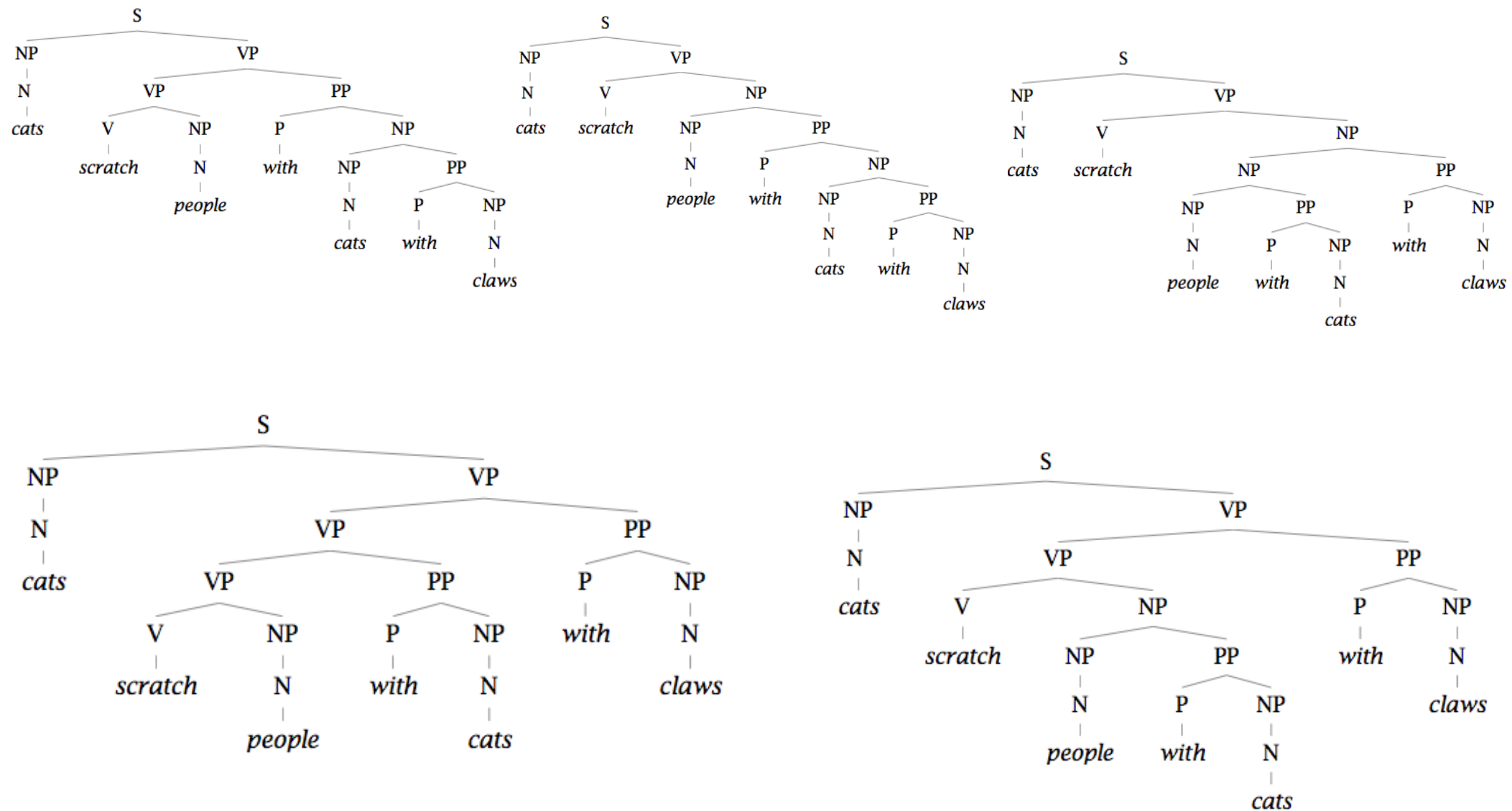
- Broad coverage

- Frequencies and distributional information

- A way to evaluate systems

Two problems to solve:

1. Repeated work...



Two problems to solve:
2. Choosing the correct parse

How do we work out the correct attachment:

She saw the man with a telescope
Is the problem 'AI complete'? Yes, but ...
Words are good predictors of attachment

Defining grammars for natural language

Context Free Grammar (CFG) - example

DT $\rightarrow$ {the, a}

N $\rightarrow$ {ball, garden, house, sushi}

P $\rightarrow$ {in, behind, with}

NP $\rightarrow$ DT N

NP $\rightarrow$ NP PP

PP $\rightarrow$ P NP

N: noun

P: preposition

NP: “noun phrase”

PP: “prepositional phrase”

Context-free grammars

A CFG is a 4-tuple $\mathbf{N}, \Sigma, \mathbf{R}, S$ consisting of:

A set of nonterminals $\mathbf{N}$

(e.g. $\mathbf{N} = \{S, NP, VP, PP, Noun, Verb, \dots\}$)

A set of terminals Σ

(e.g. $\Sigma = \{I, you, he, eat, drink, sushi, ball, \}$)

A set of rules $\mathbf{R}$

$\mathbf{R} \subseteq \{A \rightarrow \beta \text{ with left-hand-side (LHS) } A \in \mathbf{N}$

and right-hand-side (RHS) $\beta \in (\mathbf{N} \cup \Sigma)^* \}$

A start symbol $S \in \mathbf{N}$

Constituents: Heads and dependents

There are different kinds of constituents: **Noun phrases:** the man, a girl with glasses, **Prepositional phrases:** with glasses, in the garden **Verb phrases:** eat sushi, sleep, sleep soundly

Every phrase has a **head**:

Noun phrases: the man, a girl with glasses

Prepositional phrases: with glasses, in the garden

Verb phrases: eat sushi, sleep, sleep soundly

The other parts are its **dependents**. Dependents are either **arguments** or **adjuncts**

Is string α a constituent?

He talks [in class].

Substitution test:

Can α be replaced by a single word?

He talks [there].

Movement test:

Can α be moved around in the sentence?

[In class], he talks.

Answer test:

Can α be the answer to a question?

Where does he talk? - [In class].

Arguments are obligatory

Words subcategorize for specific sets of arguments:

Transitive verbs (sbj + obj): [John] likes [Mary]

All arguments have to be present:

*[John] likes. *likes [Mary].

No argument can be occupied multiple times:

*[John] [Peter] likes [Ann] [Mary].

Transitive eat (sbj + obj): [John] eats [sushi].

Intransitive eat (sbj): [John] eats.

Adjuncts are optional

Adverbs, PPs and adjectives can be adjuncts:

Adverbs: John runs [fast].

a [very] heavy book.

PPs: John runs [in the gym].

the book [on the table]

Adjectives: a [heavy] book

There can be an arbitrary number of adjuncts:

John saw Mary.

John saw Mary [yesterday].

John saw Mary [yesterday] [in town]

John saw Mary [yesterday] [in town] [during lunch]

[Perhaps] John saw Mary [yesterday] [in town] [during lunch]

Heads, Arguments and Adjuncts in CFGs

Heads:

We assume that each RHS has one head, e.g.

VP $\rightarrow$ Verb NP (Verbs are heads of VPs)

NP $\rightarrow$ Det Noun (Nouns are heads of NPs)

S $\rightarrow$ NP VP (VPs are heads of sentences)

Exception: Coordination, lists: VP $\rightarrow$ VP conj VP

Arguments:

The head has a different category from the parent:

VP $\rightarrow$ Verb NP (the NP is an argument of the verb)

Adjuncts:

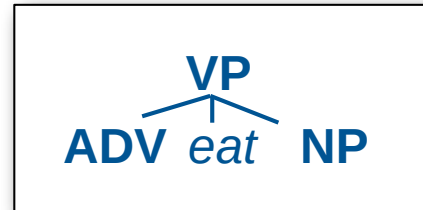
The head has the same category as the parent:

VP $\rightarrow$ VP PP (the PP is an adjunct)

Chomsky Normal Form

The right-hand side of a standard CFG can have an **arbitrary number of symbols** (terminals and nonterminals):

$VP \rightarrow ADV \text{ eat } NP$



A CFG in **Chomsky Normal Form** (CNF) allows only two kinds of right-hand sides:

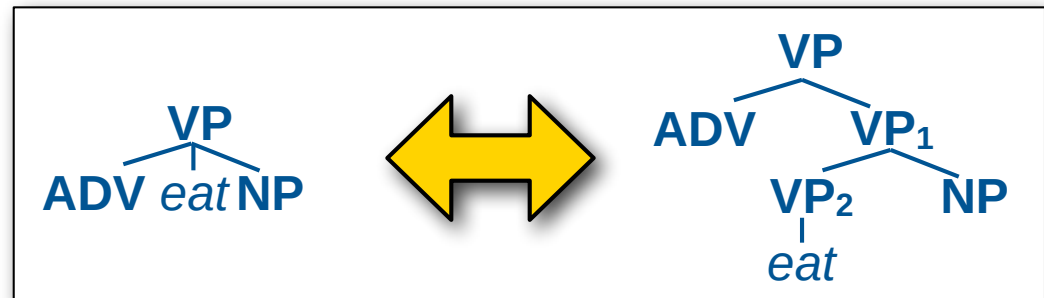
- **Two nonterminals:** $VP \rightarrow ADVVP$
- **One terminal:** $VP \rightarrow eat$

Any CFG can be transformed into an equivalent CNF:

$VP \rightarrow ADVVP \text{ } VP_1$

$VP_1 \rightarrow VP_2 \text{ } NP$

$VP_2 \rightarrow eat$



Empty(ϵ)-productions

Formally, context-free grammars are allowed to have **empty productions** (ϵ = the empty string):

$VP \rightarrow V NP$ $NP \rightarrow DT Noun$ $NP \rightarrow \epsilon$

These can always be **eliminated** without changing the language generated by the grammar:

$VP \rightarrow V NP$ $NP \rightarrow DT Noun$ $NP \rightarrow \epsilon$

becomes

$VP \rightarrow V NP$ $VP \rightarrow V \epsilon$ $NP \rightarrow DT Noun$

which in turn becomes

$VP \rightarrow V NP$ $VP \rightarrow V$ $NP \rightarrow DT Noun$

We assume that our grammars don't have ϵ -productions

CKY (Cocke-Kasami-Younger) parsing algorithm

Bottom-up parsing:

- start with the words

Dynamic programming:

- save the results in a table/chart

- re-use these results in finding larger constituents

Presumes a CFG in Chomsky Normal Form:

- Rules are all either $A \rightarrow B C$ or $A \rightarrow a$

- (with A, B, C nonterminals and a a terminal)

CKY algorithm

1. Create the chart

(an $n \times n$ upper triangular matrix for an sentence with n words)

– Each cell $\text{chart}[i][j]$ corresponds to the substring $w^{(i)} \dots w^{(j)}$

2. Initialize the chart (fill the diagonal cells $\text{chart}[i][i]$):

For all rules $X \rightarrow w^{(i)}$, add an entry X to $\text{chart}[i][i]$

3. Fill in the chart:

Fill in all cells $\text{chart}[i][i+1]$, then $\text{chart}[i][i+2]$, ...,
until you reach $\text{chart}[1][n]$ (the top right corner of the chart)

– To fill $\text{chart}[i][j]$, consider all binary splits $w^{(i)} \dots w^{(k)} | w^{(k+1)} \dots w^{(j)}$

– If the grammar has a rule $X \rightarrow YZ$, $\text{chart}[i][k]$ contains a Y
and $\text{chart}[k][j]$ contains a Z , add an X to $\text{chart}[i][j]$ with two
backpointers to the Y in $\text{chart}[i][k]$ and the Z in $\text{chart}[k][j]$

4. Extract the parse trees from the S in $\text{chart}[1][n]$.

CKY Algorithm

- Observation: any portion of the input string spanning i to j can be split at k , and structure can then be built using sub-solutions spanning i to k and sub-solutions spanning k to j .
- Meaning:
Solution to problem $[i, j]$ can be constructed from solution to sub problem $[i, k]$ and solution to sub problem $[k, j]$.

The CKY Algorithm

function CKY (no of words n , grammar P) returns table

for $i \leftarrow$ from 1 to n do

$\text{table}[i-1, i] \leftarrow \{A \mid A \rightarrow w_i \in P\}$

for $j \leftarrow$ from 2 to n do

for $i \leftarrow$ from $j-2$ down to 0 do

for $k \leftarrow i + 1$ to $j - 1$ do

$\text{table}[i,j] \leftarrow \text{table}[i,j] \cup \{A \mid A \rightarrow BC \in P, B \in \text{table}[i,k], C \in \text{table}[k,j]\}$

If the start symbol $S \in \text{table}[0,n]$ then $S \in L(G)$

CKY recognition vs. parsing

- Returning the full parse requires storing more in a cell than just a node label.
- We also require back-pointers to constituents of that node.
- We could also store whole trees, but less space efficient.
- For parsing, we must add an extra step to the algorithm: follow pointers and return the parse.

Ambiguity

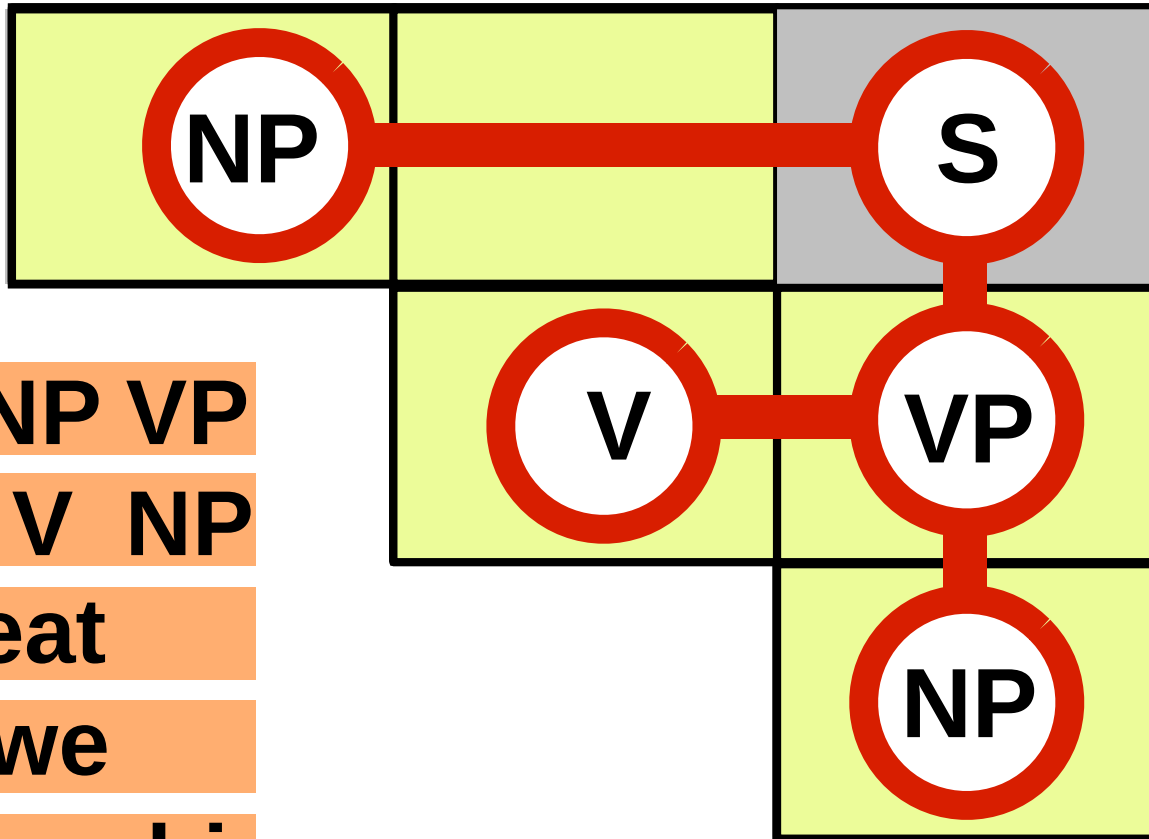
Efficient Representation of Ambiguities

- Local Ambiguity Packing :
 - a Local Ambiguity - multiple ways to derive the *same* substring from a non-terminal
 - All possible ways to derive each non-terminal are stored together
- Allows to efficiently represent a very large number of ambiguities (even exponentially many)
- Unpacking - producing one or more of the packed parse trees by following the back-pointers.

The CKY parsing algorithm

To recover the parse tree, each entry needs **pairs** of backpointers.

S → **NP VP**
VP → **V NP**
V → **eat**
NP → **we**
NP → **sushi**



We eat sushi

CKY: filling the chart

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

W	...	...	...	w_i	...	W	
							w
							...
							...
							..
							w_i
							...
							W

CKY: filling one cell

w	...		...	w_i	...	w	
							w
							...
							..
							w_i
							...
							w

chart[2][6]:

w_1 **w_2** **w_3** **w_4** **w_5** **w_6** w_7

chart[2][6]:

w_1 **w_2** **w_3** **w_4** **w_5** **w_6** w_7

w	...		...	w_i	...	w	
							w
							...
							..
							w_i
							...
							w

chart[2][6]:

w_1 **w_2** **w_3** **w_4** **w_5** **w_6** w_7

w	...		...	w_i	...	w	
							w
							...
							..
							w_i
							...
							w

chart[2][6]:

w_1 **w_2** **w_3** **w_4** **w_5** **w_6** w_7

w	...		...	w_i	...	w	
							w
							...
							..
							w_i
							...
							w

chart[2][6]:

w_1 **w_2** **w_3** **w_4** **w_5** **w_6** w_7

w	...		...	w_i	...	w	
							w
							...
							..
							w_i
							...
							w

The CKY parsing algorithm

V buy	VP buy drinks	buy drinks with	VP buy drinks with milk
	V, NP drinks	drinks with	VP, NP drinks with milk
		P	PP

Each cell may have **one entry**
for each nonterminal

We buy drinks with milk

$S \rightarrow NP VP$

$VP \rightarrow V NP$

$VP \rightarrow VP PP$

$V \rightarrow \text{drinks}$

$NP \rightarrow NP PP$

$NP \rightarrow \text{we}$

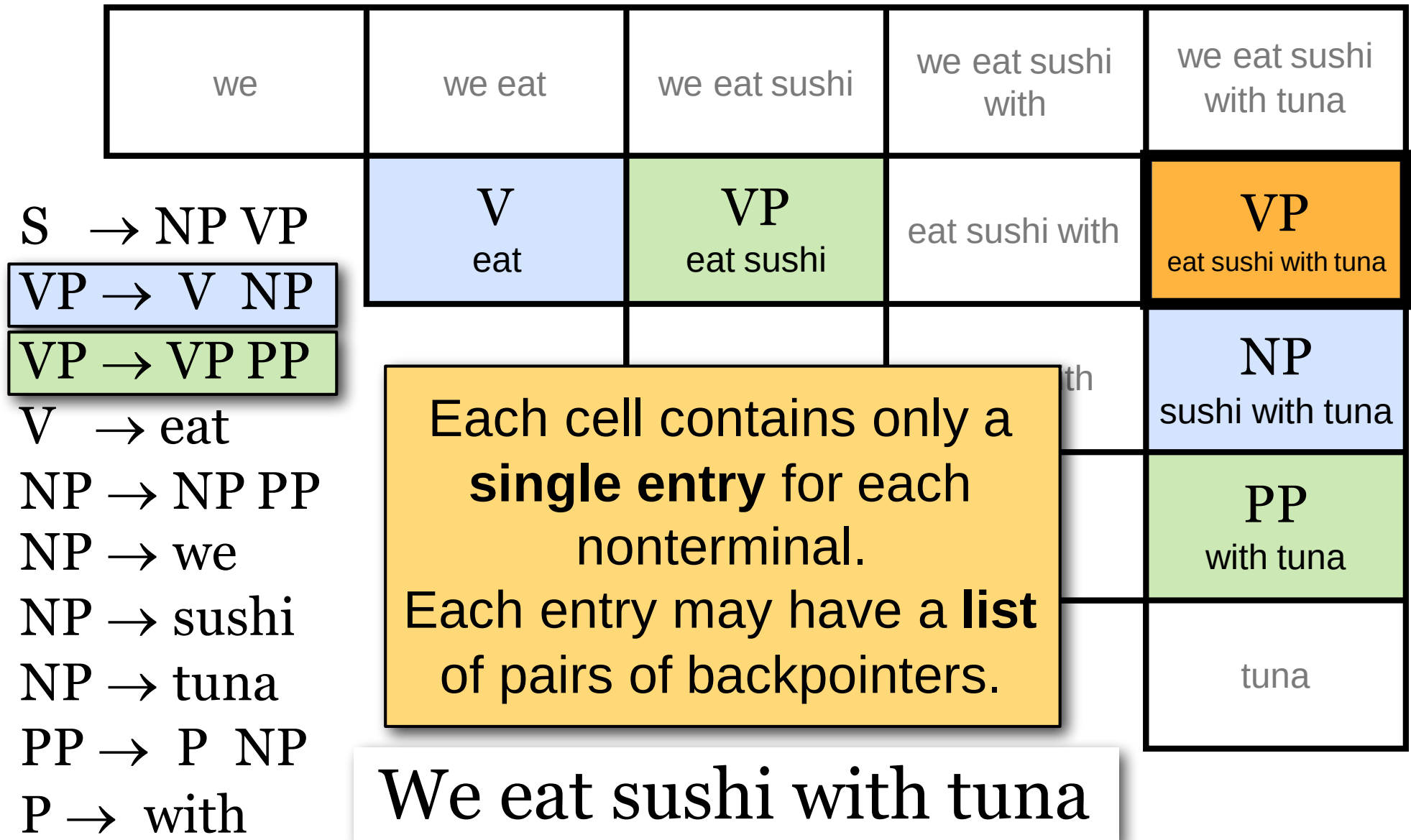
$NP \rightarrow \text{drinks}$

$NP \rightarrow \text{milk}$

$PP \rightarrow P NP$

$P \rightarrow \text{with}$

The CKY parsing algorithm



What are the terminals in NLP?

Are the “terminals”: words or POS tags?

For toy examples (e.g. on slides), it's typically the words

With POS-tagged input, we may either treat the POS tags as the terminals, or we assume that the unary rules in our grammar are of the form

POS-tag $\rightarrow$ word

(so POS tags are the only nonterminals that can be rewritten as words; some people call POS tags “preterminals”)

CKY

Each entry in a cell $\text{chart}[i][j]$ is associated with a nonterminal X .

If there is a rule $X \rightarrow YZ$ in the grammar, and there is a pair of cells $\text{chart}[i][k]$, $\text{chart}[k+1][j]$ with a Y in $\text{chart}[i][k]$ and a Z in $\text{chart}[k+1][j]$, we can add an entry X to cell $\text{chart}[i][j]$, and associate one pair of backpointers with the X in cell $\text{chart}[i][j]$

Each entry might have multiple pairs of backpointers.

When we extract the parse trees at the end,
we can get **all possible trees**.

We will need probabilities to find the single best tree!

How do you count the **number of parse trees** for a sentence?

1. For each **pair of backpointers**
(e.g. $VP \rightarrow V \ NP$): **multiply** #trees of children
$$\text{trees}(VP_{VP \rightarrow V \ NP}) = \text{trees}(V) \times \text{trees}(NP)$$
2. For each **list of pairs of backpointers**
(e.g. $VP \rightarrow V \ NP$ and $VP \rightarrow VP \ PP$): **sum** #trees
$$\text{trees}(VP) = \text{trees}(VP_{VP \rightarrow V \ NP}) + \text{trees}(VP_{VP \rightarrow VP \ PP})$$

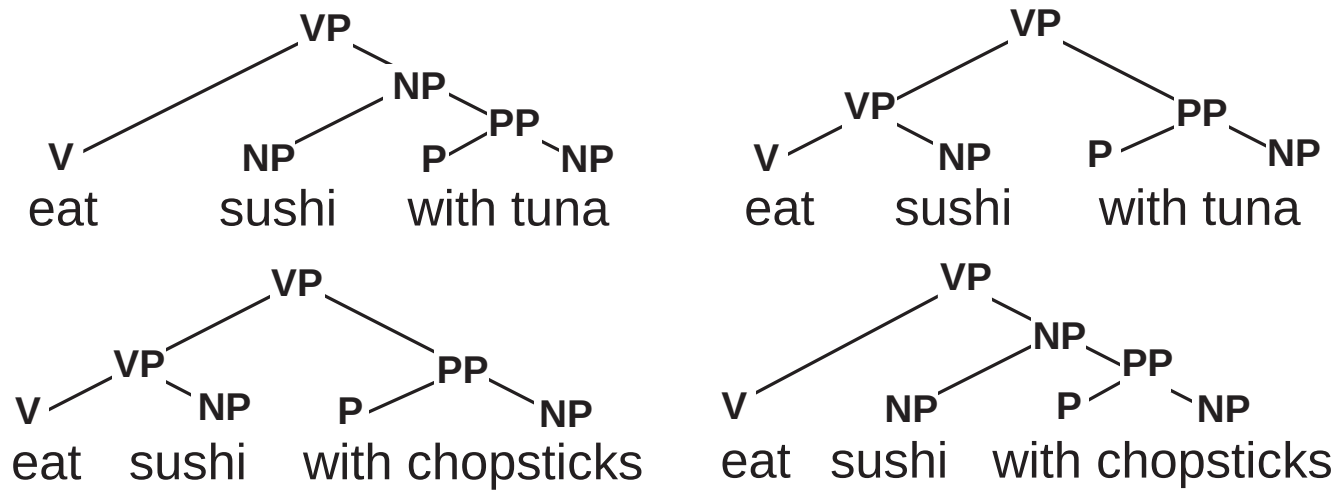
Dealing with ambiguity:

Probabilistic

Context-Free Grammars (PCFGs)

Grammars are ambiguous

A grammar might generate multiple trees for a sentence:



What's the most likely parse τ for sentence S ?

We need a model of $P(\tau \mid S)$

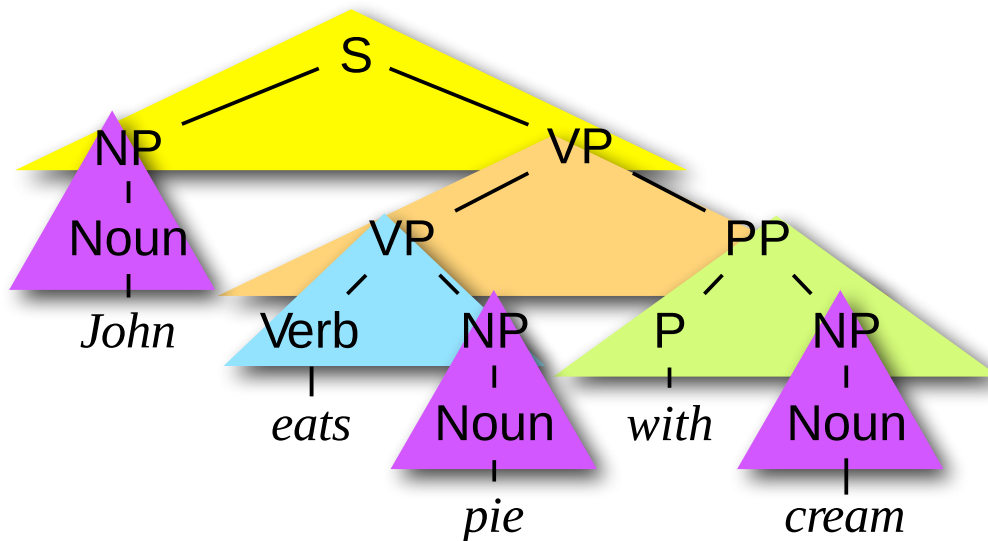
Probabilistic Context-Free Grammars

For every nonterminal X , define a probability distribution $P(X \rightarrow \alpha \mid X)$ over all rules with the same LHS symbol X :

S	NP VP	0.8
S	S conj S	0.2
NP	Noun	0.2
NP	Det Noun	0.4
NP	NP PP	0.2
NP	NP conj NP	0.2
VP	Verb	0.4
VP	Verb NP	0.3
VP	Verb NP NP	0.1
VP	VP PP	0.2
PP	P NP	1.0

Computing $P(\tau)$ with a PCFG

The probability of a tree τ is the product of the probabilities of all its rules:



$$P(\tau) = 0.8 \times 0.3 \times 0.2 \times 1.0 \times 0.2^3$$
$$= 0.00384$$

S	NP VP	0.8
S	S conj S	0.2
NP	Noun	0.2
NP	Det Noun	0.4
NP	NP PP	0.2
NP	NP conj NP	0.2
VP	Verb	0.4
VP	Verb NP	0.3
VP	Verb NP NP	0.1
VP	VP PP	0.2
PP	P NP	1.0

Example(CKY Parsing)

Context-Free Grammar

S → NP VP (0.80)

NP → Det N (0.3)

VP → V NP (0.20)

V → includes (0.05)

Det → the (0.4)

Det → a (0.4)

N → meal (0.01)

N → flight (0.02)

Sentence : the flight includes a meal

Example(CKY Parsing)

Context-Free Grammar

S -> NP VP

VP -> VBG NNS

VP-> VBZ VP

VP -> VBZ NP

NP -> DT NN

NP -> JJ NNS

Det -> a

NN ->pilot

VBZ -> likes

VBG -> flying

JJ -> flying

NNS-> planes

Sentence : a pilot likes flying planes